

1 Risk Analysis Plan

2 Security Implementation Plan

Security is integrated across all project phases and layers to ensure the chatbot platform remains safe, compliant, and resilient against real-world threats. The security design aligns with OWASP Top 10 and modern AI system protection standards.

2.1 Security Objectives

- Protect user and business data during storage, processing, and communication.
- Prevent unauthorized access or abuse through robust authentication and rate limiting.
- Ensure AI and API components cannot be exploited or injected with malicious inputs.
- Maintain system visibility and control through secure logging and monitoring.

2.2 Implementation Details

2.2.1 1. Authentication & Authorization

- Implement OAuth2 and JWT-based authentication for all users and businesses.
- Use short-lived tokens and refresh mechanisms to prevent token reuse.
- Enforce Role-Based Access Control (RBAC) for business admins and platform users.

2.2.2 2. Data Protection

- Enforce HTTPS/TLS for all communication (frontend, backend, APIs, integrations).
- Encrypt sensitive data (chat logs, credentials, business info) in PostgreSQL (pgcrypto).
- Use environment variables and secret management in CI/CD to protect API keys.

2.2.3 3. Input Validation & AI Security

- Validate and sanitize all user inputs with Pydantic schemas.
- Prevent prompt injection and ensure AI models only access intended data.
- Filter and review AI responses before displaying them to users.

2.2.4 4. API & Network Security

- Apply rate limiting and throttling on key endpoints to prevent abuse.
- Enforce CORS with domain whitelisting for integrations (WhatsApp, Facebook, Web).
- Use Docker container isolation and vulnerability scans (Trivy, Bandit).

2.2.5 5. Monitoring & Testing

- Enable structured security logging (Loguru) for authentication and API events.
- Mask sensitive data in logs.
- Automate OWASP ZAP and Bandit scans in GitHub CI/CD pipeline.
- Conduct periodic penetration testing and performance checks before release.

2.3 OWASP Top 10 Security Measures

2.3.1 1. Authentication & Session Security (A07: Identification & Authentication Failures)

Already covered: OAuth2 + JWT.

Add:

- Token rotation & short TTL (short-lived JWTs, refresh tokens stored securely).
- Device/session revocation — allow logout from all sessions.
- Secure password policy + account lockout on repeated failed logins.
- Use FastAPI's OAuth2PasswordBearer securely (no token in URL).

2.3.2 2. Access Control & Authorization (A01: Broken Access Control)

- Enforce RBAC (Role-Based Access Control) between businesses, admins, and normal users.
- Prevent access to chat histories or admin routes unless authorized.
- Use path- and object-level access control (e.g., users can only view their own chat logs).
- Validate all incoming API requests against permissions.

2.3.3 3. Data Protection & Privacy (A02: Cryptographic Failures)

- Encrypt sensitive data (e.g., chat logs, business info) in PostgreSQL using pgcrypto.
- Never store plain tokens, passwords, or API keys.
- Use `.env` for credentials (never hardcode).
- Force HTTPS for all communications, including AI API calls.

2.3.4 4. Injection & Input Validation (A03: Injection, A05: Security Misconfiguration)

- Sanitize all incoming user messages before processing (no injection into AI prompts).
- Use parameterized queries in PostgreSQL.
- Add input schema validation in FastAPI with Pydantic.
- Escape all user-provided content on the frontend (prevent XSS).
- Filter file uploads (if added later) by MIME type.

2.3.5 5. Prompt Injection & AI Layer Security (Emerging Category, OWASP LLM Top 10)

Since you're using LLMs (Gemini/Qwen):

- Implement input/output sanitization before passing data to AI.
- Restrict model access to only what's needed (no sensitive data in prompts).
- Add a safety layer that checks responses before returning them to users.
- Limit what the AI can access through LangChain (no unrestricted file or system calls).

2.3.6 6. API Security (A08: Software and Data Integrity Failures)

- Require API keys or OAuth2 for external integrations (WhatsApp, Facebook, etc.).
- Add Rate Limiting + Throttling per endpoint.
- Use CORS with strict domain whitelisting.
- Validate all payloads with schemas (avoid mass assignment vulnerabilities).

2.3.7 7. Dependency & Infrastructure Security (A06: Vulnerable & Outdated Components)

- Use Dependabot or Safety CLI to scan dependencies weekly.
- Regularly update FastAPI, React, and Hugging Face libraries.
- Keep Docker images minimal and use non-root users.
- Scan Docker images for vulnerabilities (e.g., Trivy).

2.3.8 8. Logging & Monitoring (A09: Security Logging & Monitoring Failures)

- Implement structured logging (e.g., Loguru in FastAPI).
- Log all login attempts, API errors, and suspicious activity.
- Mask sensitive data in logs (tokens, passwords).
- Set up alerting on failed login bursts or rate-limit triggers.

2.3.9 9. Server & Config Security (A05: Security Misconfiguration)

- Enforce Content Security Policy (CSP) in React.
- Disable directory listing and debug mode.
- Use secure headers (HSTS, X-Frame-Options, X-Content-Type-Options).
- Manage secrets securely (GitHub Actions Secrets + .env + Docker secrets).

2.3.10 10. Business Logic & Abuse Prevention (A10: SSRF, A04: Insecure Design)

- Validate that bots can't be abused to spam or flood messages.
- Limit AI response length and frequency.
- Validate business IDs and message origins.
- Use rate limiting per user/IP to stop abuse of free tiers or spam attacks.

Category	Risk Description	Probability	Impact	Mitigation / Prevention
Technical	AI model integration (Gemini / GPT / NileChat) may fail or produce inconsistent outputs.	Medium	High	Create a fallback layer with multiple models and strong prompt formatting.
Data Quality	Poor or insufficient Egyptian Arabic datasets for training emotion detection or intent models.	High	High	Use open datasets, fine-tuning, and real testing with SMEs.
Performance	Chatbot may experience slow response time or overload under concurrent users.	Medium	Medium	Optimize backend performance using async, caching, and monitoring tools.
Security	Unauthorized access to business data or chat logs.	Medium	High	Implement JWT, encryption, and role-based access controls.
User Acceptance	Businesses may find customization complex or prefer competitors.	Medium	High	Simplify dashboard UI and emphasize SME-friendly pricing.
Financial	Budget or sponsorship delays.	Low	Medium	Use minimal-cost deployment options and open-source resources.
Ethical / Legal	Issues with data privacy and user consent.	Low	High	Ensure consent-based data usage and anonymize logs.

Table 1: Risk Analysis Plan for the AI Customer Support Platform